Password Security & Hashing Report

Objective: The objective of this report is to understand the concept of password security using hashing techniques. This includes studying different hashing algorithms such as MD5, SHA-1, and SHA-256, and analyzing their security strengths and weaknesses. The report also aims to demonstrate the use of Linux tools like md5sum and sha256sum to generate hashes for sample passwords. Additionally, it focuses on understanding why hashing is different from encryption and identifying the key elements of a strong password policy to enhance system security.

1. What is Hashing?

Hashing is the process of converting data (like a password) into a fixed-length string called a **hash value** using a mathematical algorithm.

- It is **one-way** → you cannot reverse it back to the original password.
- Used to securely store passwords in systems.

---

2. Hashing Algorithms

🔹 **MD5 (Message Digest 5)**

- Produces a **128-bit hash**
- Fast but **insecure**
- Easily broken using brute-force and rainbow tables
  ❌ **Not recommended for password storage**

🔹 **SHA-1 (Secure Hash Algorithm 1)**

- Produces a **160-bit hash**
- More secure than MD5 but now **deprecated**
- Vulnerable to collision attacks
  ❌ **Not recommended**

🔹 **SHA-256 (Secure Hash Algorithm 256)**

- Part of SHA-2 family
- Produces a **256-bit hash**

- Strong and widely used
  Recommended (with salting)

---

3. Hashing Passwords Using Linux Tools

Step 1: Open Terminal

Step 2: Create Sample Passwords

```
echo "password123" > pass1.txt

echo "admin@123" > pass2.txt

echo "helloWorld!" > pass3.txt

echo "qwerty" > pass4.txt

echo "Secure#789" > pass5.txt
```

Step 3: Generate MD5 Hash

```
md5sum pass1.txt
```

Example Output:

```
482c811da5d5b4bc6d497ffa98491e38  pass1.txt
```

Step 4: Generate SHA-256 Hash

```
sha256sum pass1.txt
```

Example Output:

```
ef92b778bafe771e89245b89ecbcf...  pass1.txt
```

4. Why Hashing is NOT Encryption

Hashing	**Encryption**
One-way process	Two-way process
Cannot be reversed	Can be decrypted with a key
Used for password storage	Used for secure communication

Example:

- Hashing: Store password safely in database
- Encryption: Send secure messages (like HTTPS)

---

5. What Makes a Strong Password Policy?

A strong password policy includes:

Length

- At least **12–16 characters**

Complexity

- Uppercase letters (A–Z)
- Lowercase letters (a–z)
- Numbers (0–9)
- Special characters (!@#$%)

Avoid Common Passwords

- No “123456”, “password”, “qwerty”

Use Unique Passwords

- Different password for each account

Password Expiry (Optional)

- Change periodically (e.g., every 90 days)

Account Lockout

- Lock account after multiple failed attempts

Multi-Factor Authentication (MFA)

- Adds extra security layer (OTP, biometrics)

---

6. Conclusion

- MD5 and SHA-1 are outdated and insecure.
- SHA-256 is stronger but should be used with **salting**.
- Hashing protects passwords because it is **irreversible**.
- Strong password policies are essential to prevent attacks.